



Machine Learning

part 1 :

Supervised learning on MNIST

**ATTEINDRE UNE ACCURACY DE 0.97 TOUT EN
MINIMISANT LE TEMPS D'ENTRAÎNEMENT**

Powered by

Models



```
class SimpleMLP(nn.Module):
    def __init__(self):
        super().__init__()

        self.model = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.model(x)
```

SimpleMLP :
784 → 128 → 64 → 10
ReLU()
~ 14 sec

```
class SecondeMLP(nn.Module):
    def __init__(self):
        super().__init__()

        self.model = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.model(x)
```

SecondeMLP :
784 → 64 → 10
Plus simple
~15 sec

```
class ThirdMLP(nn.Module):
    def __init__(self):
        super().__init__()

        self.model = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.model(x)
```

ThirdMLP :
784 → 128 → 64 → 10
BatchNorm()
~ 6 sec

```
class FourthMLP(nn.Module):
    def __init__(self):
        super().__init__()

        self.model = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 10)
        )

    def forward(self, x):
        return self.model(x)
```

FourthMLP :
784 → 64 → 32 → 10
Compromis profondeur / légèreté
~17 sec

Complexité :

- Plus de couches = plus de capacité
- Mais plus lent à entraîner

1.SimpleMLP

- Bon équilibre
- 2.SecondeMLP
- Moins précis
 - plus de temps

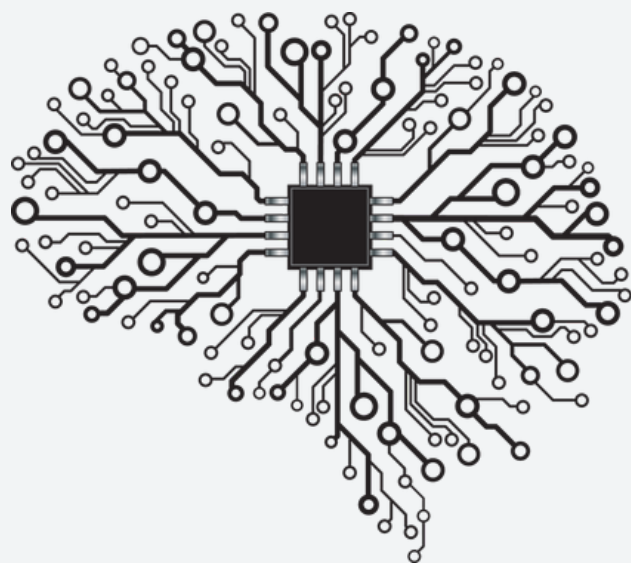
3.ThirdMLP (BatchNorm)

- Convergence plus rapide
- 4.FourthMLP
- Plus lent sans gain réel

Impact du Learning Rate

Le learning rate contrôle la vitesse d'apprentissage du modèle.

Un taux trop élevé ralentit la convergence et rend l'entraînement instable



Learning Rate	Epochs pour atteindre 0.97	Temps total	Observation
0.001	4 epochs	~15 sec	Stable, bon équilibre
0.01	10 epochs	~36 sec	Instable, convergence lente
0.005	2 epochs	~7 sec	Très rapide ⚡ (non reproductible)

Le résultat avec 0.005 est non reproductible, ce qui m'a amené à introduire des Optimisations techniques

Optimisations techniques

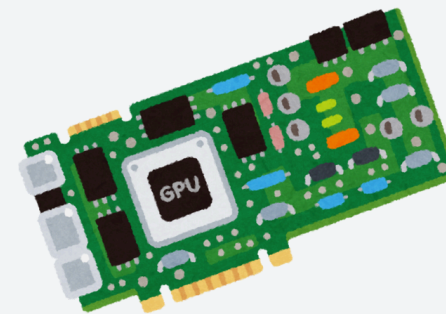
mise en Place d'une seed :

- Reproductibiliter des test
- Comparaisons entre les test

42

GPU vs CPU

- gain elever avec un Batch Size elever



Batch size (32 → 512) :

- si batch size elever sur le CPU : + de temp
- si Batch Size faible sur le GPU gain de temp faible



Adam vs SGD

Optimizer	Fonctionnement	Avantage	Inconvénient
SGD	Mise à jour simple du gradient	Léger, stable	Lent à converger
Adam	Adaptive + momentum	Très rapide	Plus complexe

- SGD met à jour les poids en suivant directement le gradient à chaque étape.
- Adam adapte automatiquement le learning rate pour chaque paramètre et utilise une mémoire des gradients passés (momentum).



Réduction des évaluations

- evaluation chronaphages
- boost l'entraînement

Résultat clé & Conclusion

OBJECTIF ATTEIND EN 6 SECONDES, DEUX FOIS PLUS RAPIDE QUE LES PREMIÈRES VERSIONS

MEILLEUR SETUP :

- ThirdMLP (BatchNorm)
- GPU
- batch_size = 512
- Adam

Résultat :

- Accuracy : 0.9731
- Temps : 6.15 sec

Batch size = facteur clé

BatchNorm = accélère convergence

Adam > SGD

Optimisation globale > modèle seul

le temps d'entraînement passe de 14 sec à 6 sec

Ce projet montre que l'optimisation des paramètres d'entraînement est souvent plus importante que la complexité du modèle.

part 2.1 :

Supervised learning

Classification

Data Set

- 2 feature [label (spam/ham)/ message]
- 5572 samples => ham = 4825 + spam = 747

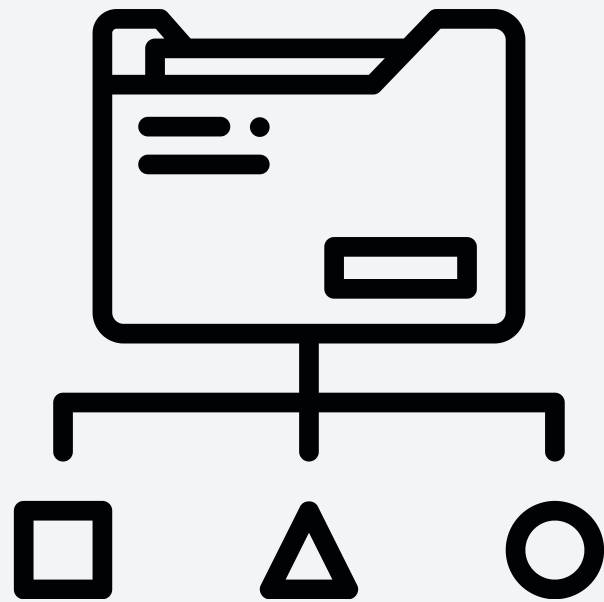


SMS Spam Collection
SMS Spam Collection Data Set
[kaggle.com](https://www.kaggle.com/abhi4jain/sms-spam-collection-data)

5572 exemple de sms en anglais, a qui leur est assosier un label correspondant a si le message est un spam ou non.

	Label	Message
5567	Spam	This is the 2nd time we have tried 2 contact u...
5568	Ham	Will Ã¼ b going to esplanade fr home?
5569	Ham	Pity, * was in mood for that. So...any other s...
5570	Ham	The guy did some bitching but I acted like i'd...
5571	Ham	Rofl. Its true to its name

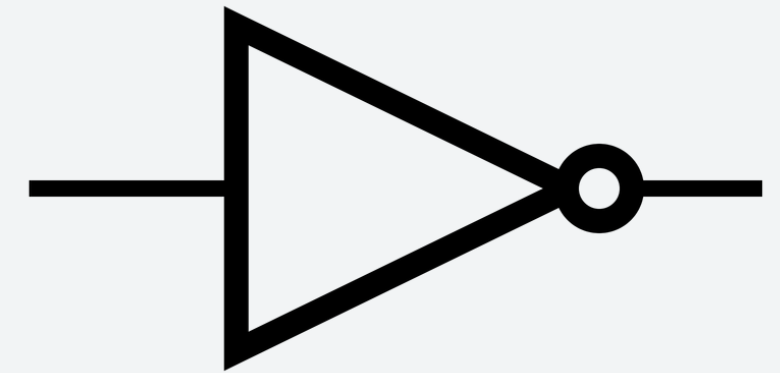
Objectif



IA en machine learning de classification



verifier si un SMS est un Spam ou non (Spam / ham). .



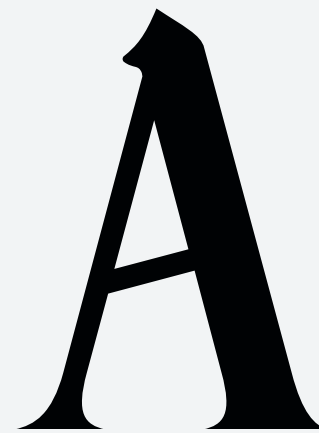
Classement binaire

Remarque



dataset est fortement déséquilibré :

- 87% messages normaux (ham).
- 13% pour les spam.



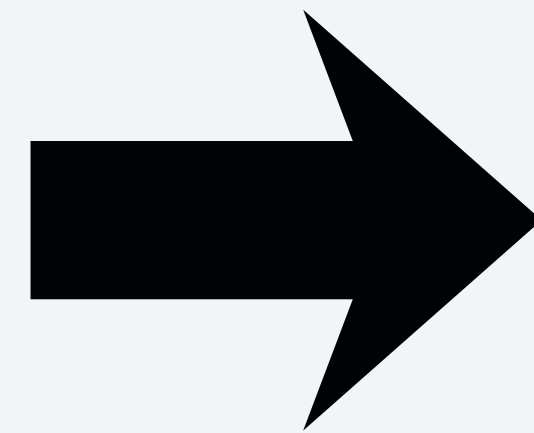
suppression des:

- majuscule
- ponctuation
- ...

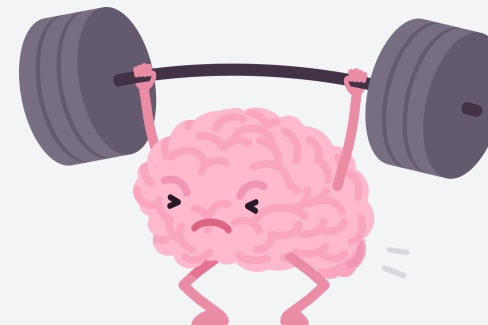


label simplifier :

- HAM = 0
- Spam = 1



les messages sont vectorisés
afin de mesurer l'importance
des mots dans chaque SMS



séparation du data set en 2 :

- entraînement et test
- afin d'éviter d'effectuer des tests biaisés, comme le set de données n'est pas équilibré, il est nécessaire d'utiliser l'option stratify dans `train_test_split()`.

Uniter de Mesure

une comparaison entre train accuracy et test accuracy afin de verifier si le model apprend bien.

si $\text{test_accuracy} \approx \text{train_accuracy}$:

- le model apres bien.
- sinon le model memorise et n'apprend pas vraiment.

1. Recall → mesure de performance couramment utilisée pour les modèles de classification.

2. Precision → éviter faux positifs

3. F1-score → équilibre des deux

Premier resultat obtenue

	Predit Ham	Predit Spam
Vrai Ham	966	0
Vrai Spam	44	105

	Precision	Recall	f1-score
Ham (0)	0.96	1.00	0.98
Spam (1)	1.00	0.70	0.83

Models Multinomial Naïve Bayes

1. calcule quelle est la probabilité que ce message soit un spam, étant donné les mots qu'il contient ?
2. Concrètement :
 - compte la fréquence des mots dans spam vs ham
 - Il compare les probabilités
3. Avantages :
 - Très rapide
 - Très efficace pour le texte
 - Peu de paramètres

```
MultinomialNB(alpha=0.05)
```

alpha :

- il arrive qu'un mot n'apparaisse jamais dans les données d'entraînement.
- probabilité = 0 → et comme on multiplie les probabilités → tout devient 0
- Le lissage (alpha) évite ça en ajoutant une petite valeur à tous les comptes.

Conclusion:

~ 6% des spam envoyés sont considérés
comme des ham pour seulement 0.7% de
Ham qui sont détectés comme Spam

	Prédit HAM	Prédit SPAM
Vrai HAM	959	7
Vrai SPAM	9	140

Classification Report :

	precision	recall	f1-score	support
HAM (0)	0.99	0.99	0.99	966
SPAM (1)	0.95	0.94	0.95	149
Accuracy			0.99	1115
macro avg	0.97	0.97	0.97	1115
weighted avg	0.99	0.99	0.99	1115

Train accuracy : 0.9982050706753421 Test accuracy : 0.9856502242152466

Model Logistic Regression

1. Modèle linéaire qui combine les mots avec des poids pour prédire la probabilité qu'un message soit un spam.

2. Concrètement :

- Chaque mot influence la décision
- Le modèle apprend automatiquement les poids

3. Avantages :

- Plus flexible que Naive Bayes
- Meilleur équilibre precision/recall
- Pas d'hypothèse d'indépendance

Chaque mot a un poids :

- "free" → +3 (spam)
- "win" → +2 (spam)
- "hello" → -1 (ham)

conclusion:

- le model detect plus que 7% de Spam en Ham
- les Spam non detecter ne sont pas les meme que le model precedant

	Prédit HAM	Prédit SPAM
Vrai HAM	961	4
Vrai SPAM	11	138

Classification Report :

	precision	recall	f1-score	support
HAM (0)	0.99	0.99	0.99	966
SPAM (1)	0.97	0.93	0.95	149
Accuracy			0.99	1115
macro avg	0.98	0.96	0.97	1115
weighted avg	0.99	0.99	0.99	1115

Train accuracy : 0.9928202827013687 Test accuracy : 0.9856502242152466

Model Voting Classifier

1. Combiner plusieurs modèles pour améliorer les performances
2. Concrètement :
 - Utilise plusieurs modèles
 - Combine leurs prédictions
3. Avantages :
 - Plus robuste
 - Réduit les erreurs
 - Exploite les forces de chaque modèle

```
VotingClassifier(  
    estimators=[  
        ('nb', model_nb),  
        ('lr', model_lr)  
    ],  
    voting='soft'  
)
```

Voting = 'soft' :
le modèle combine les probabilités prédites
par chaque classifieur, plutôt que leurs
simples votes.

Conclusion:

- Le model est interessant
- peut etre plus performant avec plus de model different.

	Prédit HAM	Prédit SPAM
Vrai HAM	965	1
Vrai SPAM	12	137

Classification Report :

	precision	recall	f1-score	support
HAM (0)	0.99	1.00	0.99	966
SPAM (1)	0.99	0.92	0.95	149
Accuracy			0.99	1115
macro avg	0.98	0.96	0.97	1115
weighted avg	0.99	0.99	0.99	1115

Train accuracy : 0.9988781691720888 Test accuracy : 0.9883408071748879

Conclusion

phase de preprocessing incluant le nettoyage des données et la vectorisation, plusieurs modèles ont été testés : Multinomial Naive Bayes, Logistic Regression et une approche par ensemble (Voting).

Les modèles individuels ont montré d'excellentes performances, avec une accuracy proche de 99%. Logistic Regression a permis d'obtenir un meilleur recall, tandis que Naive Bayes offrait une précision légèrement supérieure.

Afin de tirer parti des différences entre ces modèles, une approche de type Voting a été mise en place. Le voting "soft", basé sur les probabilités, a permis d'obtenir un modèle plus équilibré.

Le modèle final atteint une accuracy de 98.8%, avec un recall de 92% sur les spams et une précision de 99%, ce qui en fait un modèle fiable et performant.

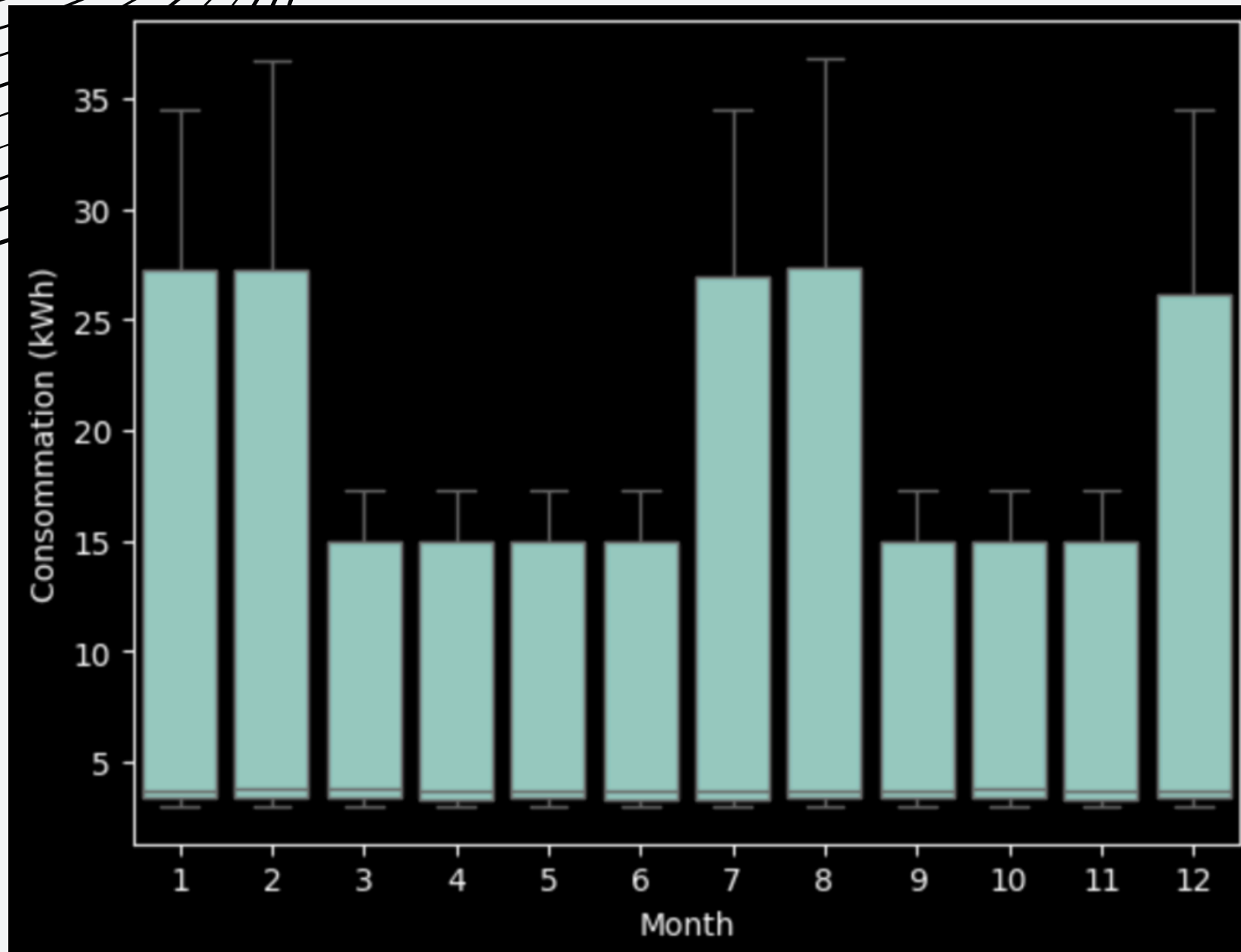
part 2.2 :

Supervised learning

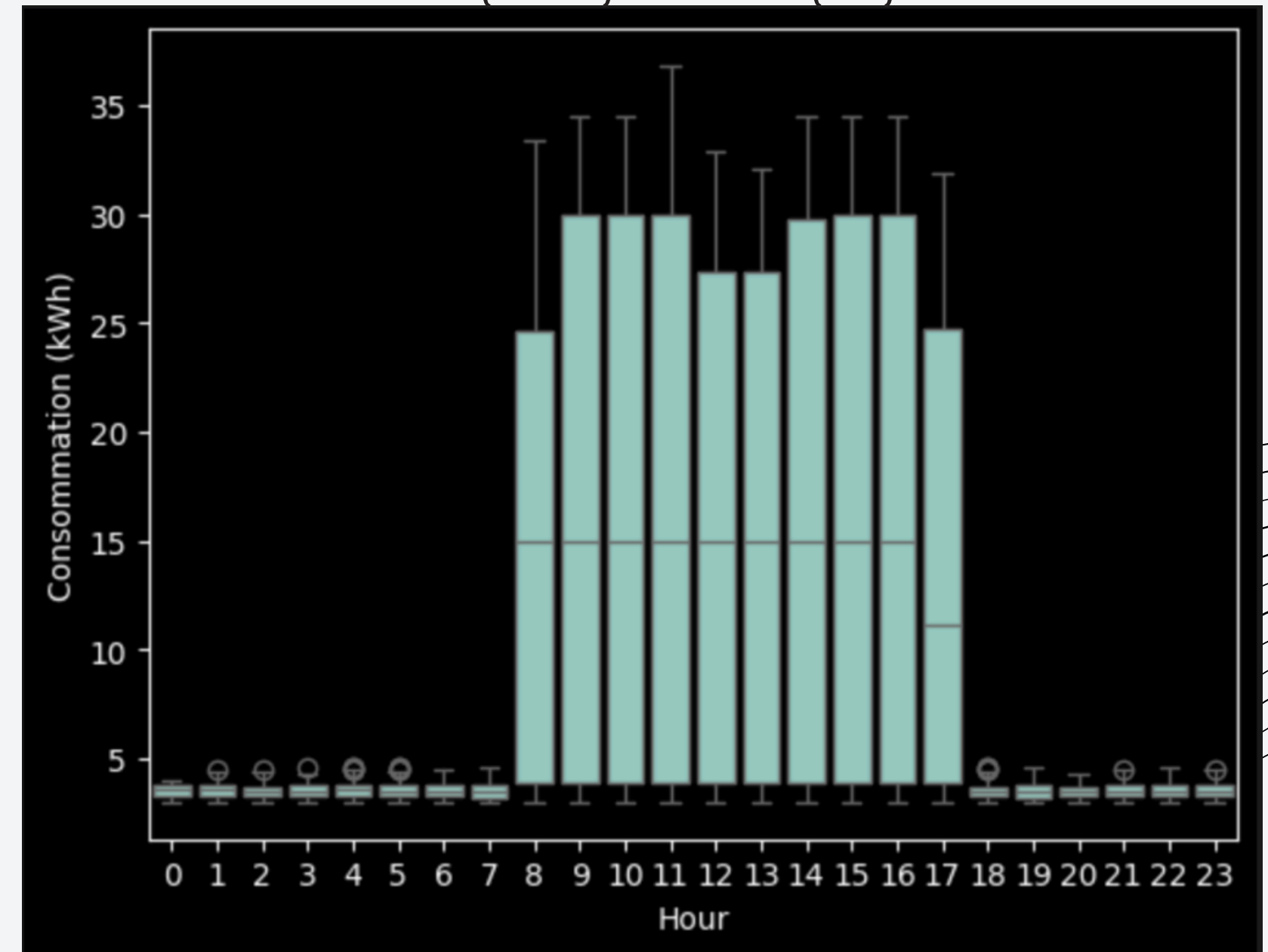
Regression

Data Set

- 5 Feature : Consommation (kWh); State; Month; Day; Hour;
- 8761 samples : du 1er Janvier 2023 au 31 decembre 2023



- La consommation energetique au pat horraire d'un batiment exprimer en Kwh.
- L'etat du batiment (ouver / Fermer),
- date (MM/DD) et de l'heure (24H).



Consomation tres cyclique :

- En fonction des saison
- En fonction de L'heure

Objectif

prédire la consommation énergétique d'un bâtiment en fonction du temps et de son état (ouvert/fermé). Dans un but d'optimisation énergétique / réduction des coûts.

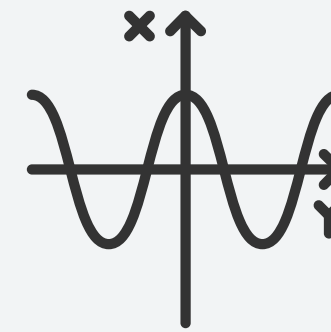
	Consommation (kWh)	State	Month	Day	Hour
0	3.374540	Fermé	1	1	0
1	3.751280	Fermé	1	1	1
2	3.616274	Fermé	1	1	2
3	3.035687	Fermé	1	1	3
4	3.170338	Fermé	1	1	4

Normalisation de la données

0/1

state symplifier :

- Fermé = 0
- ouvert = 1



normalisation de l'heure et du
mois en fonction du cosinus et
sinus

l'heure allant de 0 à 23 il est possible que le model les considere tres different alors que se sont des valeur tres proche au niveau de l'heure.
de meme pour les mois allant de 1 a 12, le model vois des valeur tres eloigner, mais en realiter janvier est juste apres decembre.

separation du data set en 2 :

- entraînement
- test

Uniter de Mesure

1. R^2 (coefficient de détermination)
 - a quel point ton modèle est meilleur qu'un modèle qui prédit toujours la moyenne de consommation
 - plus R^2 est proche de 1 $\rightarrow$ mieux
2. MAE (Mean Absolute Error)
 - erreur moyenne entre ta prédiction et la vraie valeur.
 - $MAE = 0.37 \rightarrow$ ton modèle se trompe en moyenne de 0.37 kWh
 - plus MAE est faible $\rightarrow$ mieux

Linear Regression

MAE: 2.5194604709726818

R2: 0.7985364850292678

Temps : 0.0029993057250976562

Powered by

Model Linear Regression

1. Description

- Modèle simple qui suppose une relation linéaire entre les variables d'entrée et la cible.
- Il cherche une droite (ou un plan) qui minimise l'erreur.

2. Avantages

- Très rapide à entraîner
- Interprétable (on comprend l'effet de chaque variable)
- Fonctionne bien si la relation est vraiment linéaire
- Peu de paramètres à régler

3. Limite

- Mauvais si les relations sont non linéaires

conclusion :

- MAE est tres elever
- R^2 test $\approx$ R^2 Train ne sont pas overfitting
- R^2 asser eloigner de 1.
- model non adapter: modèle linéaire le problème est non linéaire (cycles, saisons, ouverture...)

Linear Regression

MAE: 2.5194604709726818

Train R^2 : 0.8050917647139294

Test R^2 : 0.7985364850292678

Temps : 0.0019998550415039062

Model Random Forest

1. Description

- Ensemble de plusieurs arbres de décision.
- Chaque arbre fait une prédiction, et on prend la moyenne.

2. Avantages

- Gère bien les relations non linéaires
- Robuste au bruit et au surapprentissage
- Peu de prétraitement nécessaire
- Peut gérer des données complexes

3. Limite

- Moins interprétable
- Plus lent que la régression linéaire

conclusion :

- MAE est faible
- $R^2_{\text{test}} \approx R^2_{\text{Train}}$ ne sont pas overfitting
- R^2 proche de 1.
- des prediction qui suivent bien la realiter
- un grand nombre d'erreur mais avec peu de kWh de difference
- erreur moyenne = MAE $\rightarrow$ 0.33 kWh

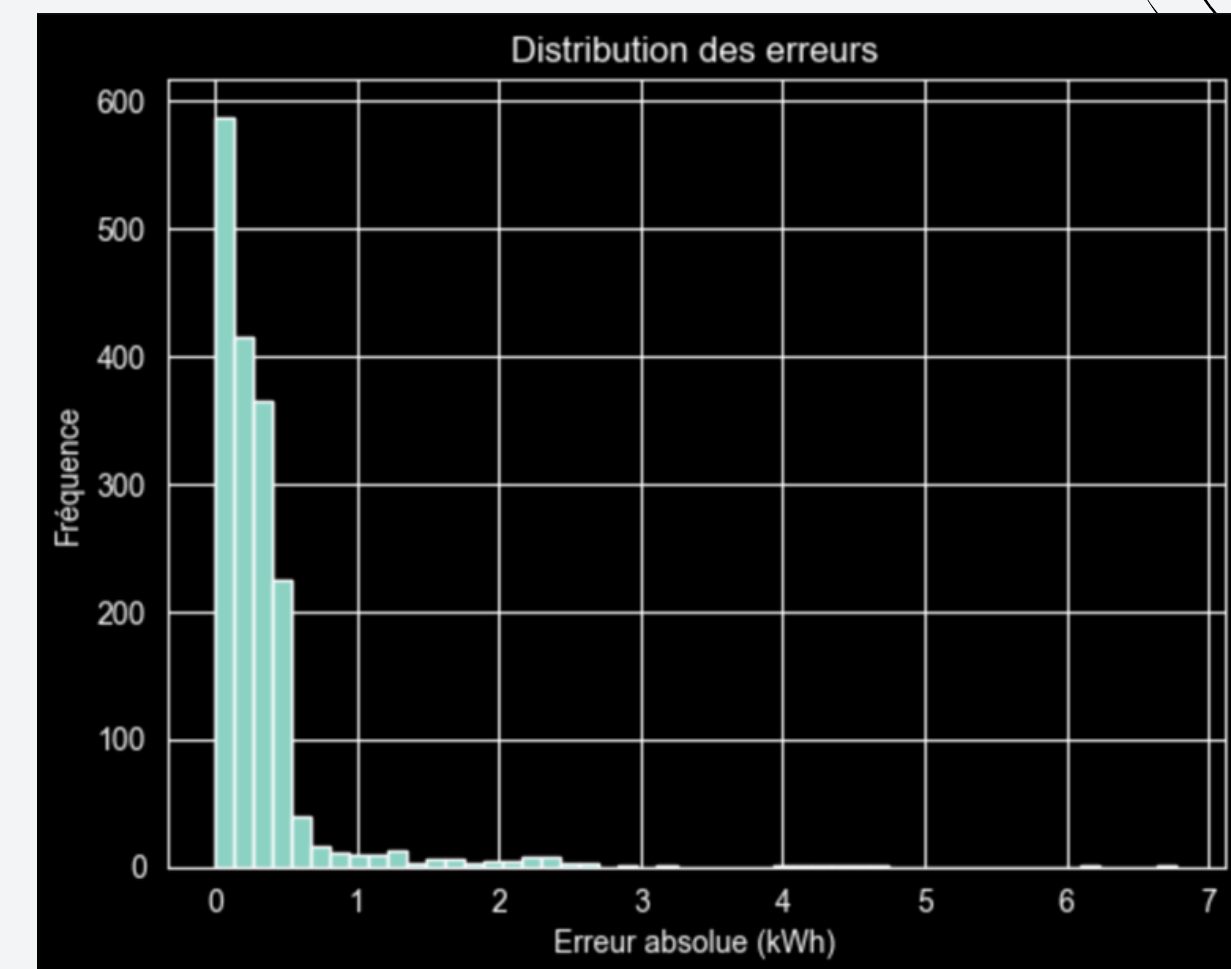
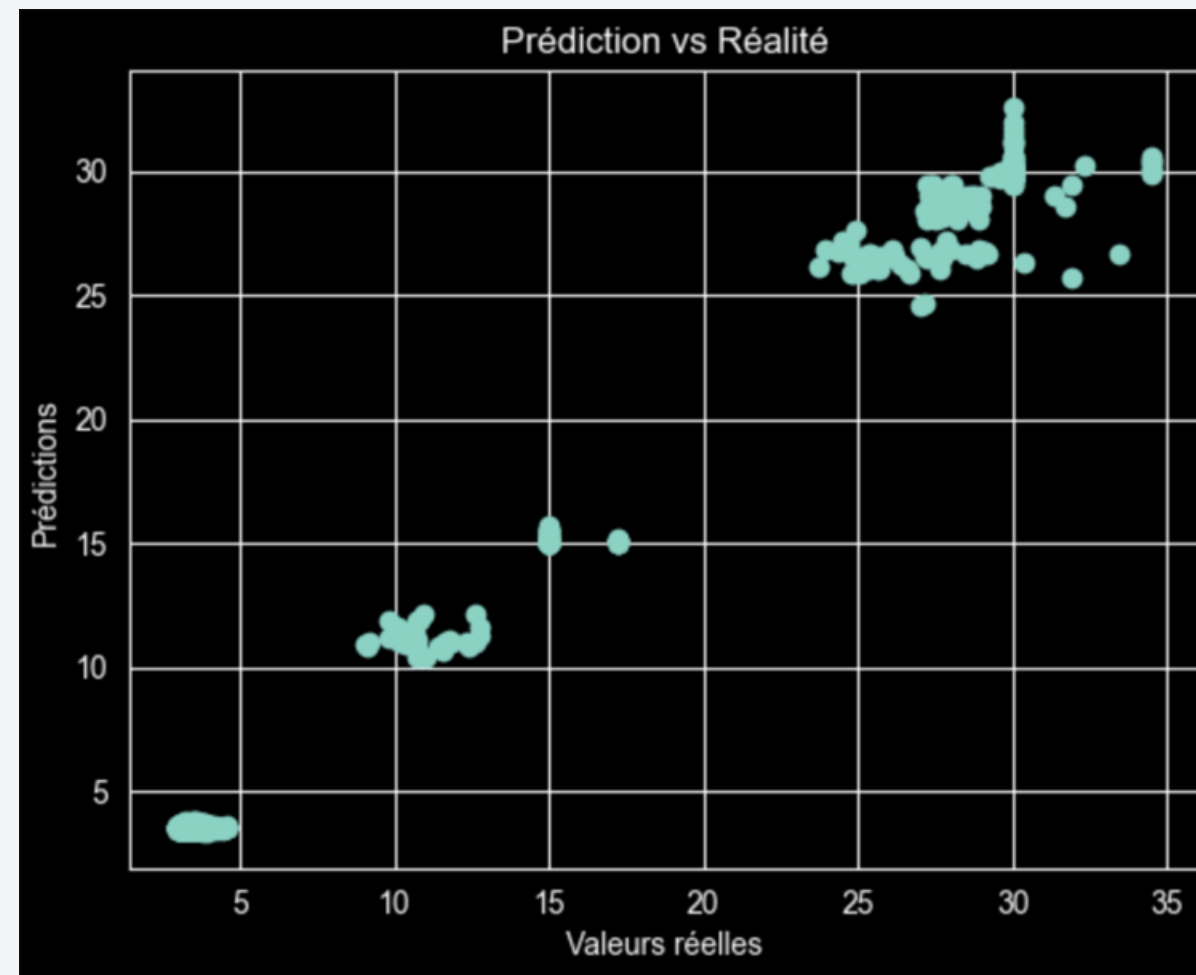
Random Forest

MAE: 0.33607198645118835

Train R^2 : 0.9963088242958525

Test R^2 : 0.9950974659608134

Temps : 0.03193521499633789



Gradient Boosting

1. Description

- Construit des arbres séquentiellement :
- chaque nouveau modèle corrige les erreurs du précédent.

2. Avantages

- Très performant (souvent meilleur que Random Forest)
- Capte des relations complexes

3. Limite

- Plus lent à entraîner
- Sensible aux paramètres (peut surapprendre si mal réglé)

Gradient Boosting

MAE: 0.3554101999568904

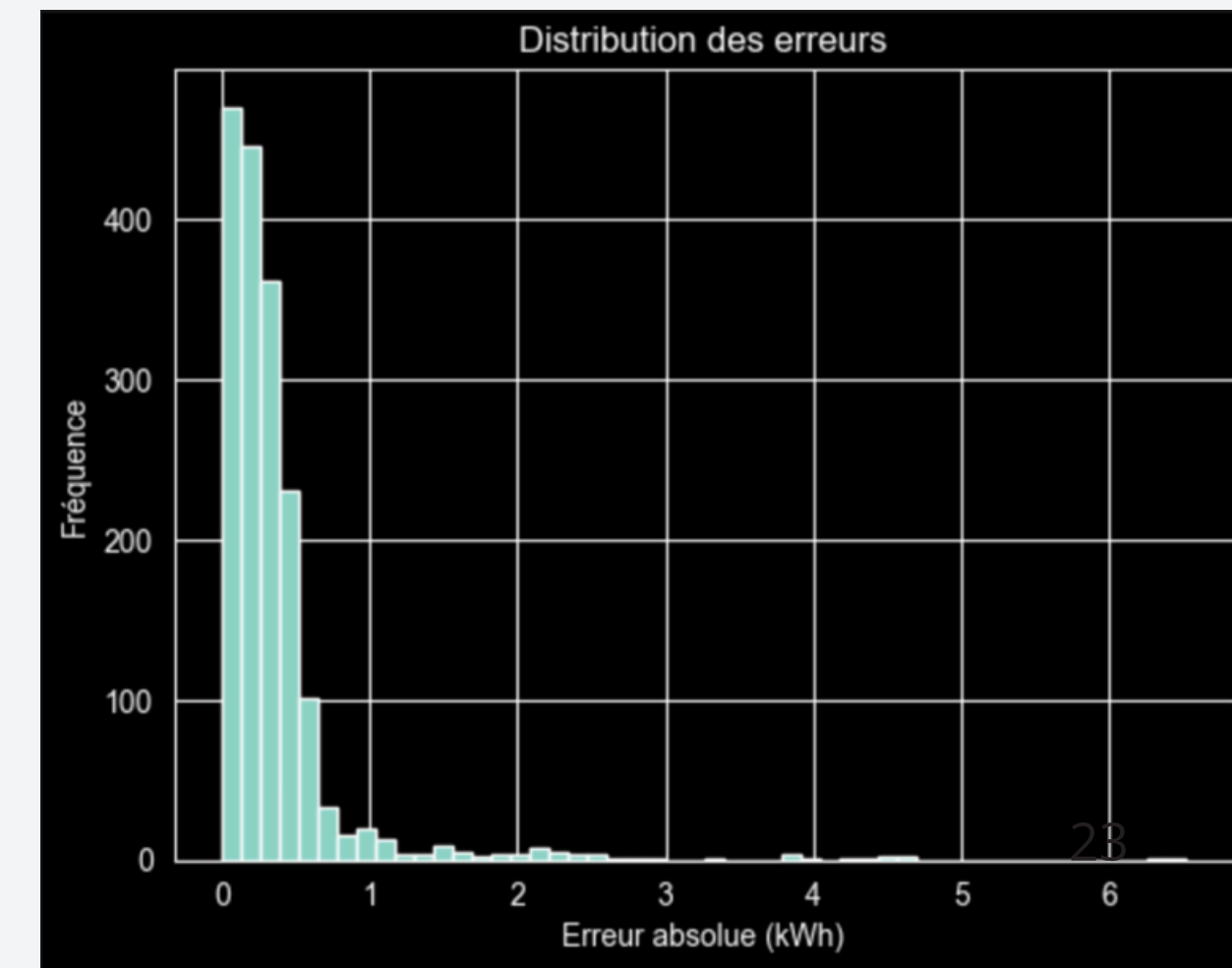
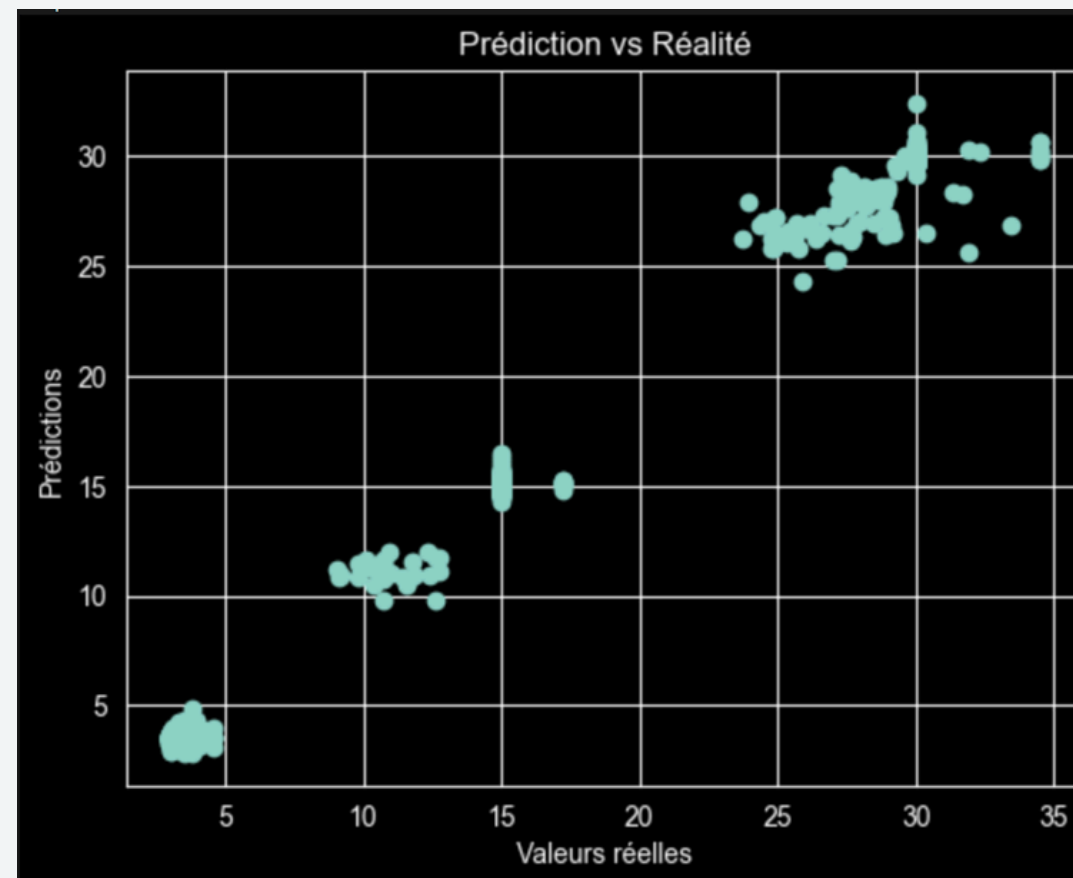
Train R2: 0.9968547505126149

Test R2: 0.9950672211123257

Temps : 0.3406665325164795

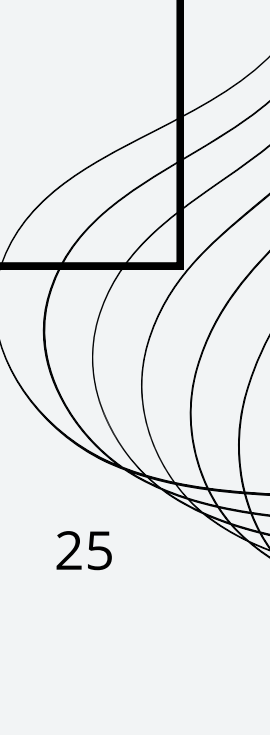
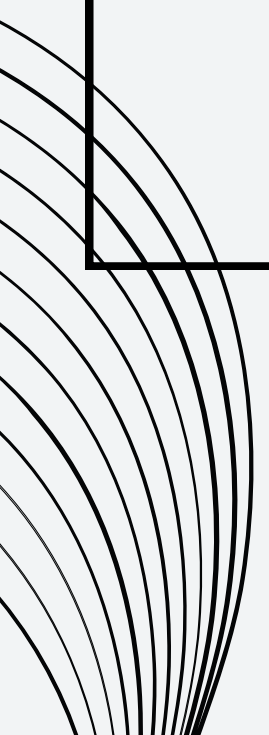
conclusion :

- MAE est faible
- R2 test $\approx$ R2 Train ne sont pas overfitting
- R2 proche de 1.
- des prediction qui suivent bien la realiter
- un grand nombre d'erreur mais avec peu de Kwh de difference
- erreur moyenne = MAE $\rightarrow$ 0.35 Kwh



Conclusion

- Les modèles non linéaires (Random Forest et Gradient Boosting) donnent de très bonnes performances avec un R^2 proche de 1 et une faible erreur.
 1. Random Forest est plus de 10 fois plus rapide que Gradient Boosting (0.03 $\rightarrow$ 0.3)
 2. ces deux modèles ont des R^2 test ou entraînement très proches l'un de l'autre pas d'overfitting
 3. le MAE de Random Forest est un peu plus petit que gradient Boosting



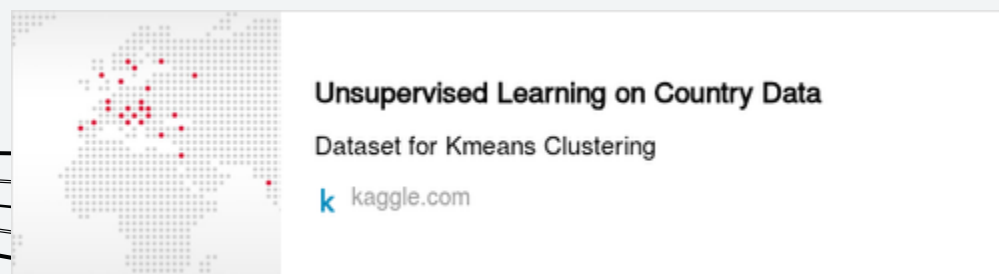
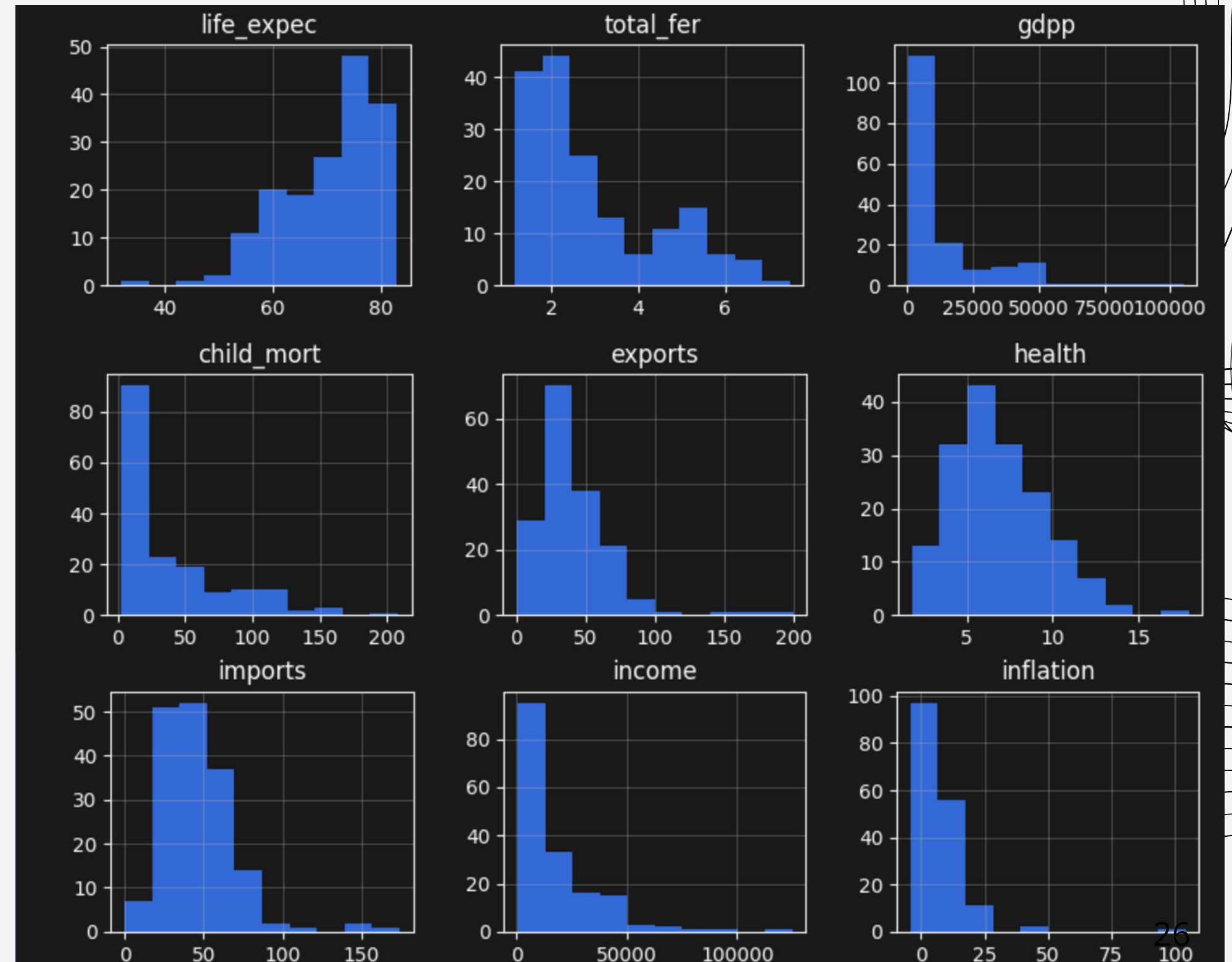
part 2.3 :

NON Supervised learning

Clustering

DATA SET

1. Ce dataset contient des informations socio-économiques sur 167 pays en 2010. (samples)
2. 1 feature texte (country)
3. 9 feature numériques décrivant différents aspects du développement d'un pays :
 - **child_mort** : nombre de décès d'enfants de moins de 5 ans pour 1000 naissances
 - **exports** : exportations (% du PIB)
 - **health** : dépenses de santé (% du PIB)
 - **imports** : importations (% du PIB)
 - **income** : revenu moyen par habitant
 - **inflation** : taux de croissance annuel
 - **life_expec** : espérance de vie
 - **total_fer** : nombre moyen d'enfants par femme
 - **gdpp** : PIB par habitant



Objectif

- Je cherche à regrouper les pays selon leur niveau de développement
- Le clustering permettra d'identifier des profils de pays (développés, en développement, etc.) sans utiliser de variable cible.
- Cluster 1 : pays riches
- Cluster 2 : pays en développement
- Cluster 3 : pays pauvres

Se baser sur une seule variable comme le GDP (pib par habitant) peut être trompeur, car le développement d'un pays dépend de plusieurs facteurs comme la santé, l'éducation et les conditions de vie.
Le clustering permet de prendre en compte plusieurs dimensions simultanément.

Normalisation et Uniter de Mesure

Une variable dont les valeurs vont de 0 à 1 000 000 (ex: le Revenu) écrasera totalement une variable qui va de 0 à 10 (ex: le taux de chômage).

$$z = \frac{x - \mu}{\sigma}$$

- μ = moyenne de la colonne
- σ = écart-type
- z = valeur transformée

1. PCA :

- technique de réduction de dimension pour passer d'un tableau a 9 dimation actuellement a un tableau en 2 dimmension

2. silhouette_score(X_scaled, clusters) :

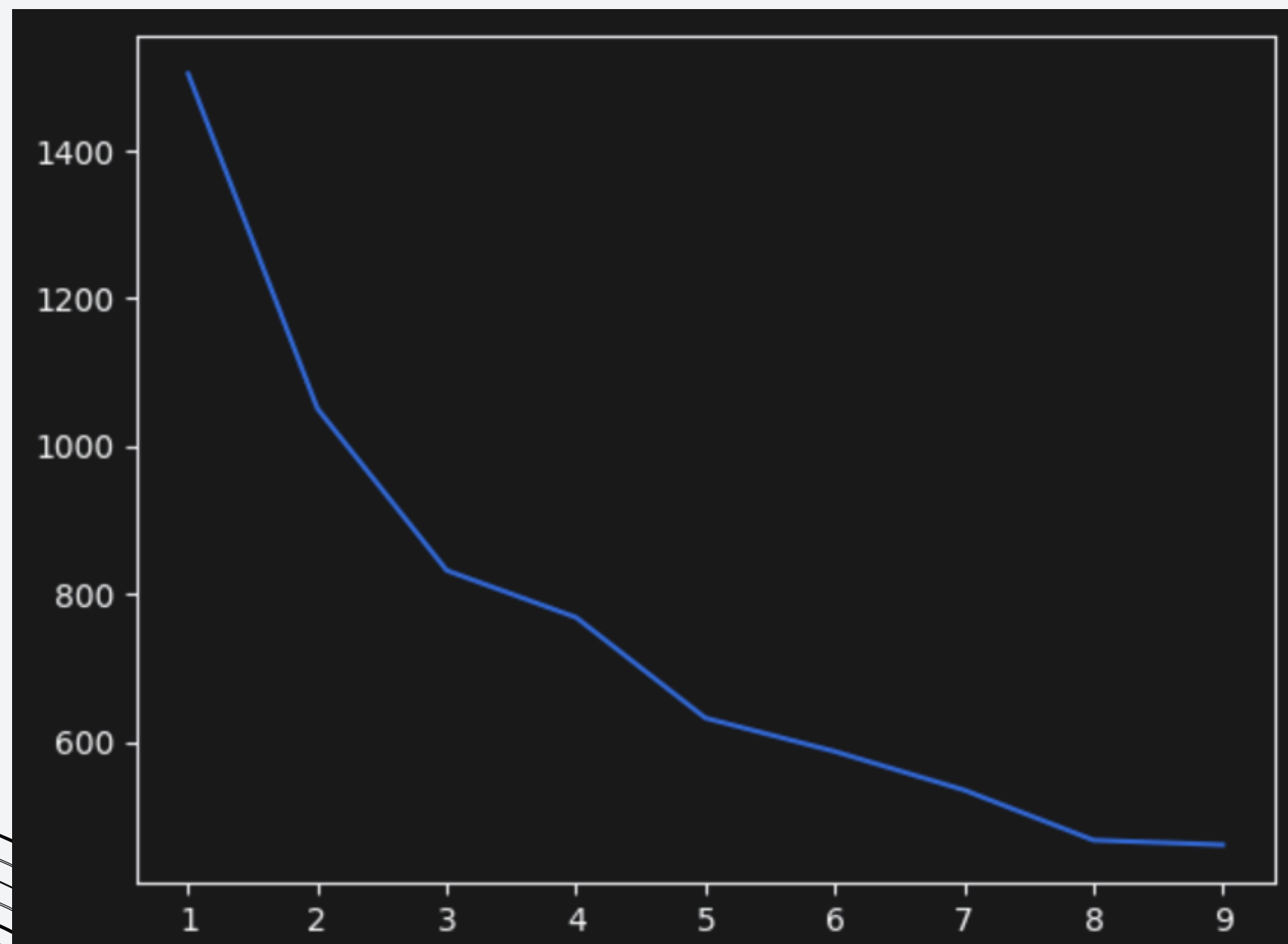
- Proche de 1 → Les clusters sont denses et très bien séparés.
- Proche de 0 → Les clusters se chevauchent ou la distance entre eux est très faible.
- Négatif → Des points ont probablement été affectés au mauvais cluster.

3. df.groupby("cluster").mean()

- la moyenne de chaque variable pour chaque groupe d'individus
- profil type de chaque cluster

méthode du coude et silhouette_score

apprentissage non supervisé pour
déterminer le nombre optimal de clusters
k à utiliser avec l'algorithme K-Means.



silhouette_score

k = 2 → 0.2662961111870726
k = 3 → 0.285600988953231
k = 4 → 0.2880471307804802
k = 5 → 0.30088229124112015
k = 6 → 0.23565028812238528
k = 7 → 0.2479313491087983
k = 8 → 0.20198237114728412
k = 9 → 0.19230727869997855
k = 10 → 0.19624516158796698

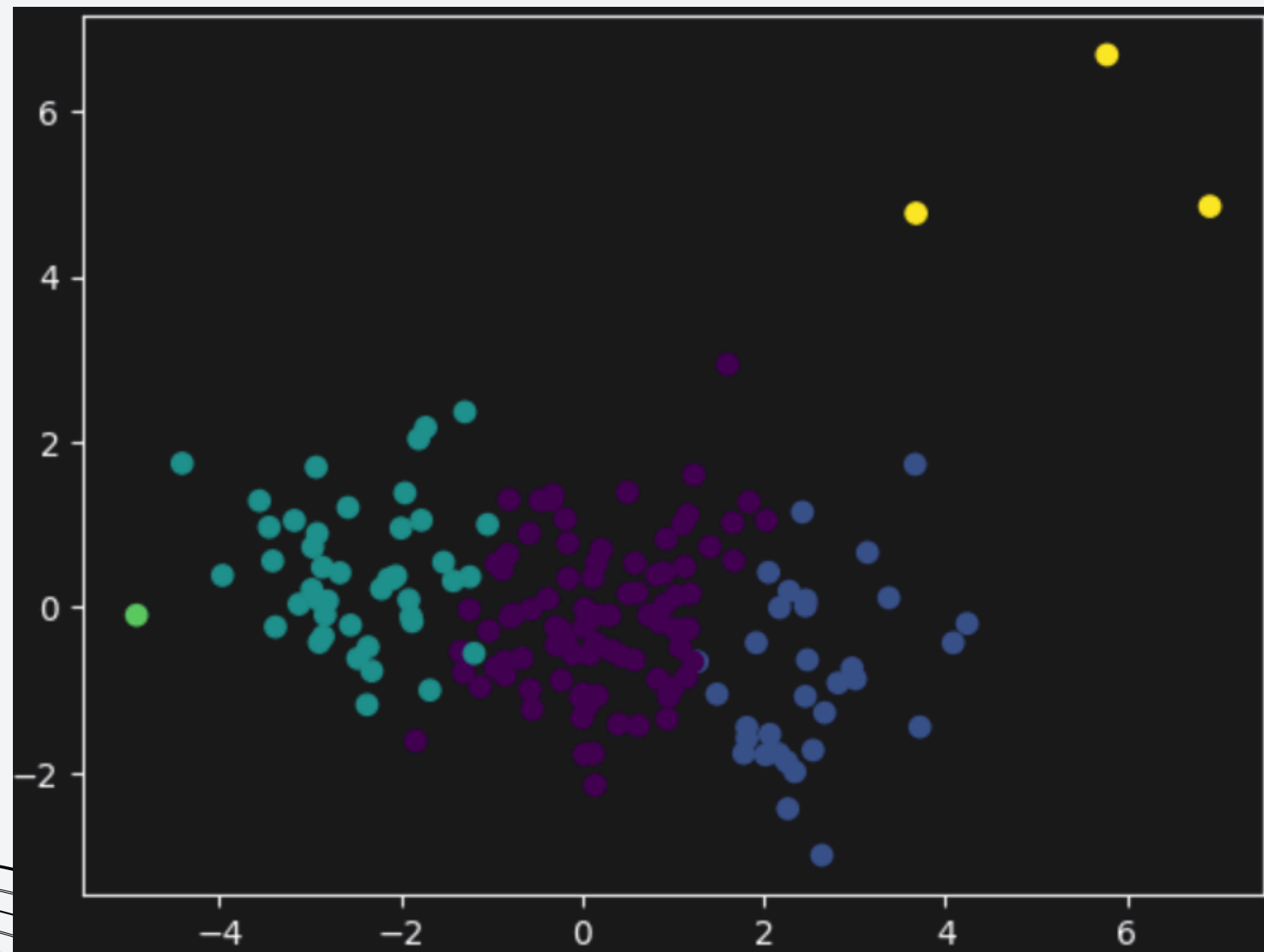
K optimal = 3 ou 5 :

- methode du coude
- silhouette_score
- coherence avec le projet

Model KMeans

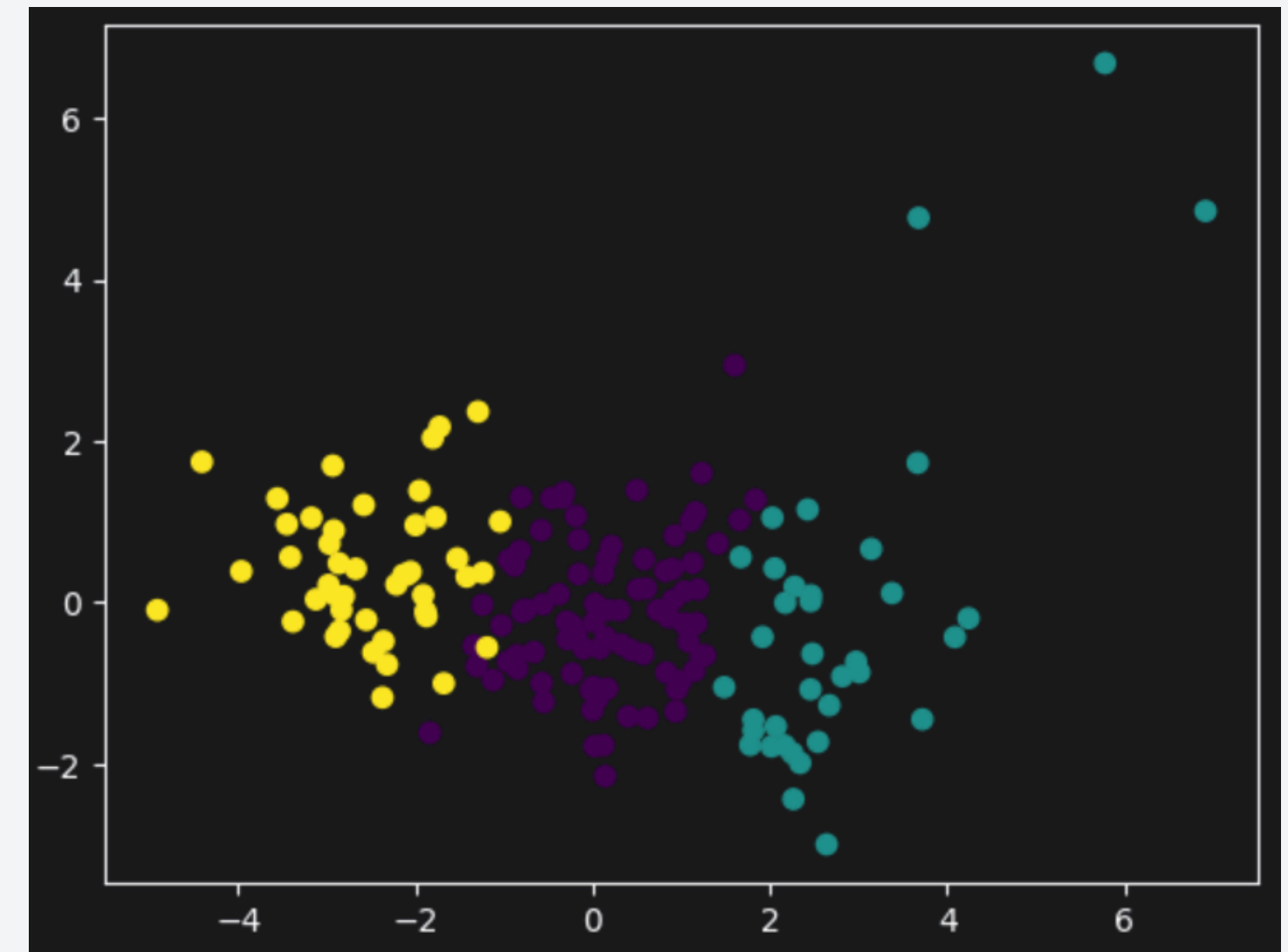
partitionner les données en K groupes (clusters) en minimisant la distance entre les points et le centre de leur groupe (appelé centroïde).

on crée k (5) groupes autour de centres



silhouette_score = 0.30088229124112015

on crée k (3) groupes autour de centres

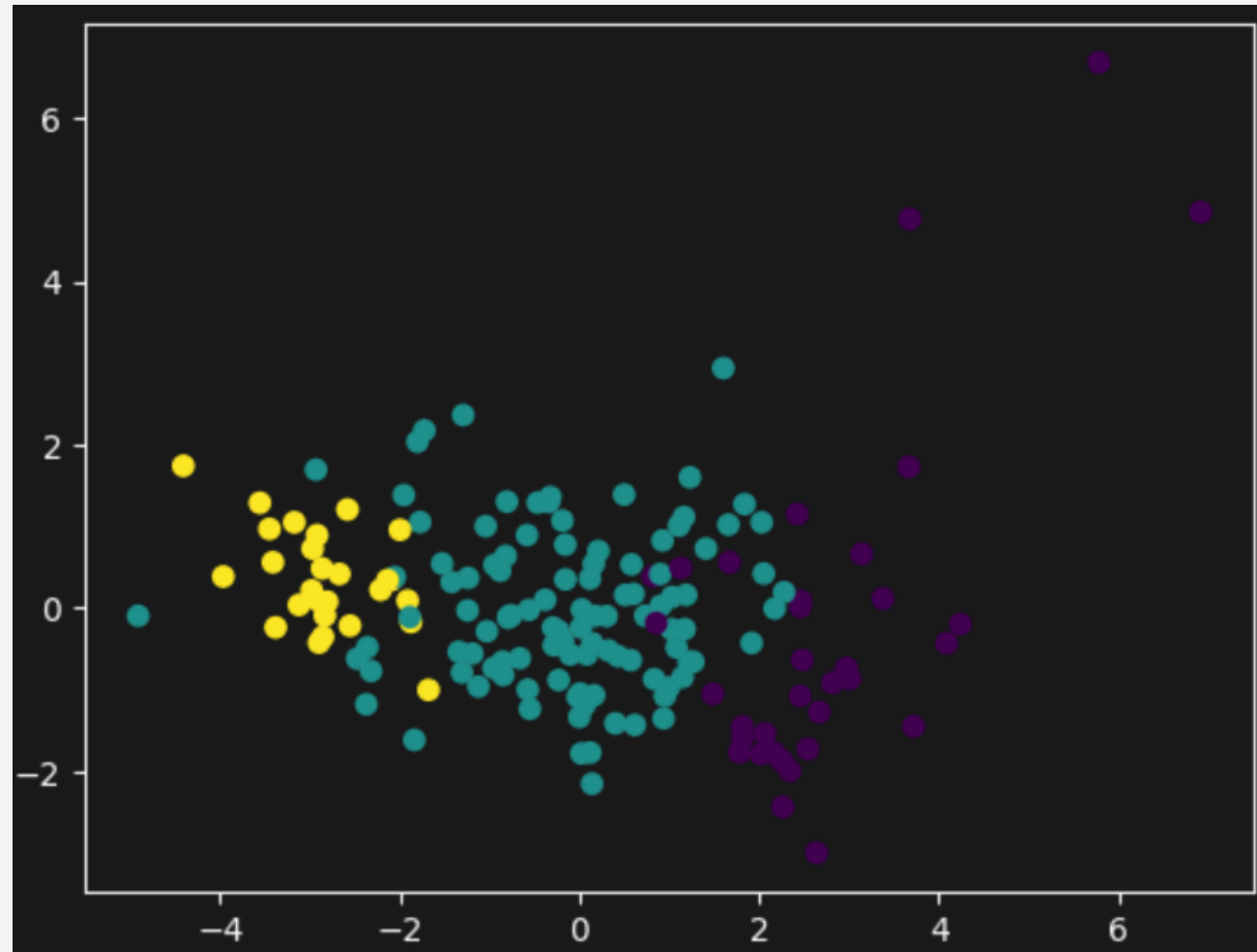


silhouette_score = 0.285600988953231

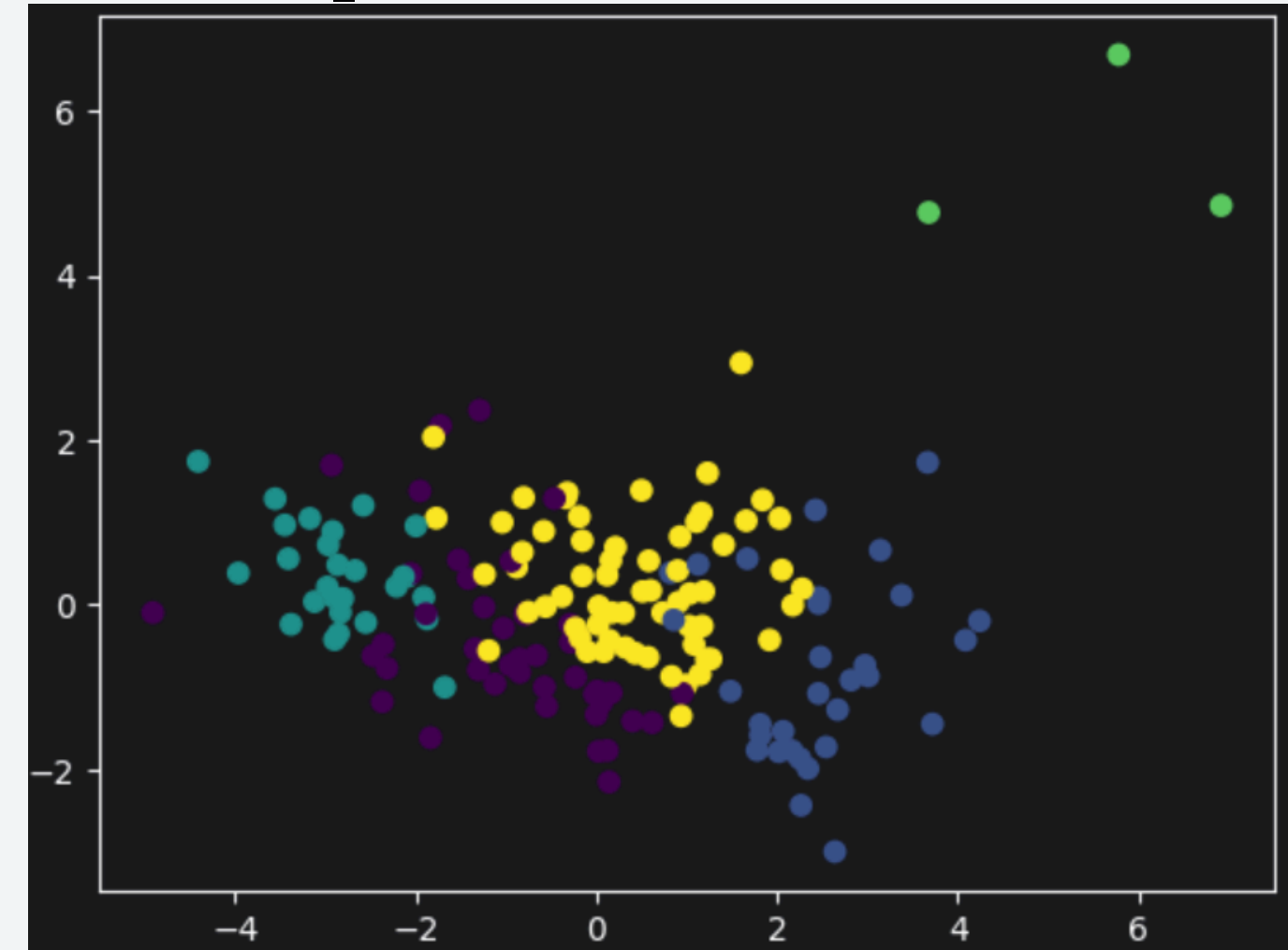
Model Agglomerative Clustering

construit les clusters en fusionnant progressivement les points selon une distance donnée

silhouette_score = 0.24563001303300652



silhouette_score = 0.21933015989396873

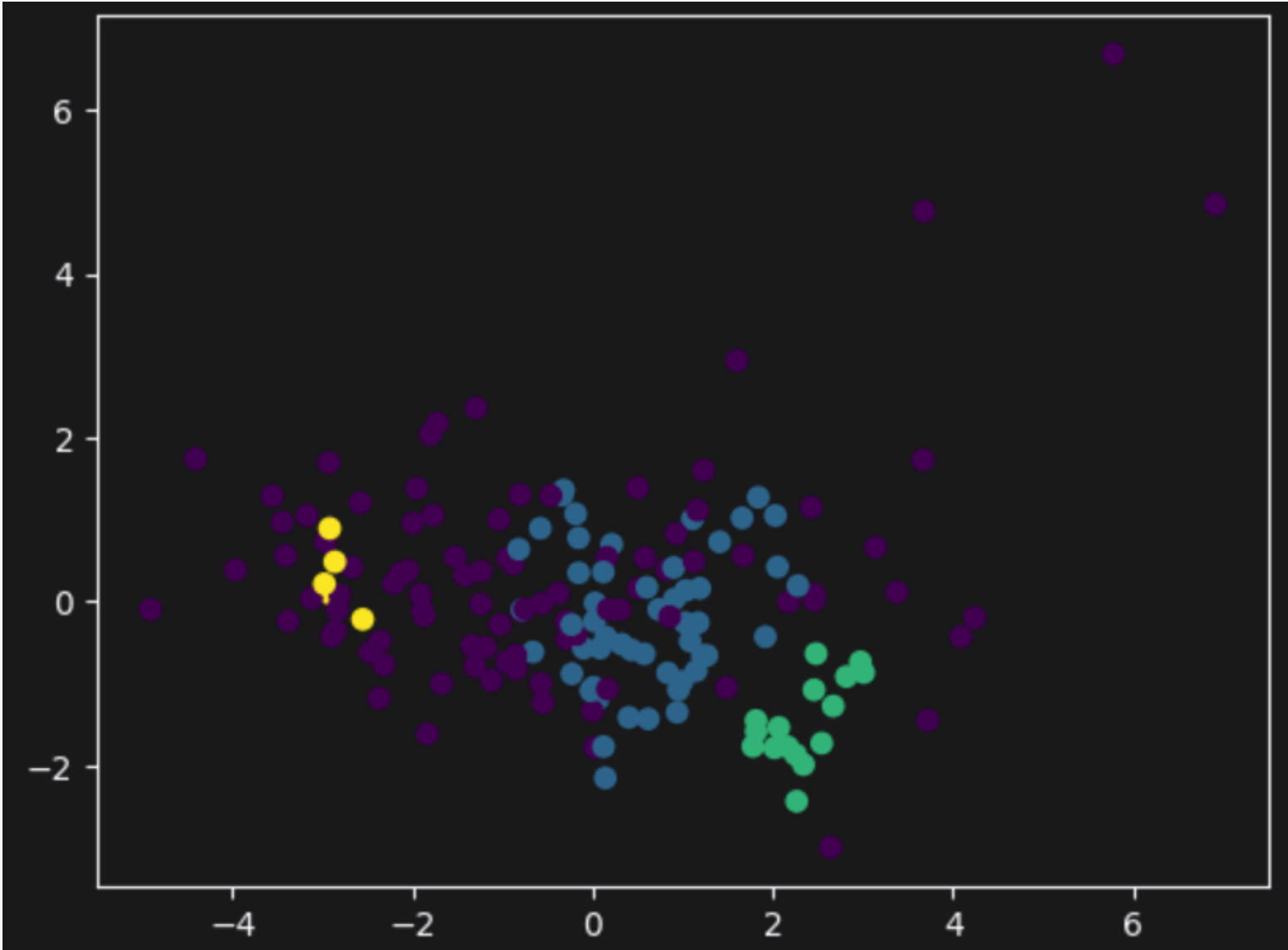


conclusion :

- la visualisation montre des cluster moins bien repartie qu'avec le model Kmeans, avec des point se trouvant proche de nuage d'autre cluster.
- le silhouette_score confirme la visualisation avec un score plus petit qu'au test precedent
- Agglomerative clustering produit des clusters similaires à KMeans mais avec une séparation légèrement moins bonne.

Model DBSCAN

regroupe les points en fonction de leur densité et permet d'identifier des valeurs aberrantes.



0.5	[-1]
1.0	[-1 0 1 2]
1.5	[-1 0]
2.0	[-1 0]

le seul cas interesant est EPS = 1.0 car cet
le seul a repere plusieurs cluster

	child_mort	exports	health	imports	income	inflation	life_expec	total_fer	gdpp
cluster_dbscan									
-1	53.218085	43.187223	6.266596	48.023041	16667.819149	11.038447	67.552128	3.517660	11462.287234
0	17.036538	41.371154	6.814808	50.117308	13278.076923	3.607385	74.263462	2.050192	8157.038462
1	4.125000	33.393750	10.219375	32.775000	37312.500000	1.371125	81.006250	1.720000	41237.500000
2	87.340000	24.000000	6.256000	37.200000	1785.600000	10.486000	55.020000	5.504000	718.600000

silhouette_score = -0.024818901891032592

conclusion :

- Une grande partie des données est classée comme bruit (cluster -1),
- Le modèle ne parvient pas à structurer correctement l'ensemble du dataset.

Conclusion

1. KMeans:

- 3 clusters
- Niveaux de développement des pays.

2. DBSCAN :

- obtient un score silhouette légèrement supérieur
- **Ne fournit pas une segmentation exploitable**

3. AgglomerativeClustering :

- obtien malheureusement de mon bon score que KMeans en étant très proche .

4. k=3 offre une segmentation simple des pays en trois niveaux de développement.

5. k=5 permet une analyse plus fine en distinguant des groupes spécifiques comme les pays très riches ou les pays en situation de crise.

0. Pays intermédiaires / Pays en développement

income $\approx$ 12k

life_expec $\approx$ 72

Amérique du Sud, Europe de l'Est...

1. Pays riches classiques / Pays développés

income $\approx$ 44k

life_expec $\approx$ 80

child_mort $\approx$ 5

Europe, USA, etc.

2. Pays pauvres / Pays en difficulté

income $\approx$ 3500

life_expec $\approx$ 59

child_mort $\approx$ 94

Afrique subsaharienne, etc.

3. Cas extrêmes

child_mort $\approx$ 130 (!!)

inflation $\approx$ 104 (!!)

income $\approx$ 5150

Pays en crise / instables

4. Ultra riches / outliers économiques (les point en haut a droite)

income $\approx$ 64k

exports $\approx$ 176 (!!)

imports $\approx$ 156 (!!)

gdpp $\approx$ 57k

Luxembourg, Singapour, Qatar...

	child_mort	exports	health	imports	income	inflation	life_expec	total_fer	gdpp
cluster									
0	22.219540	41.083782	6.246667	47.795010	12678.965517	7.727747	72.625287	2.333333	6494.390805
1	5.181250	46.118750	9.088437	40.584375	44021.875000	2.513844	80.081250	1.788437	42118.750000
2	94.313636	28.677500	6.329091	42.872727	3503.250000	9.895568	59.022727	5.047727	1753.909091
3	130.000000	25.300000	5.070000	17.400000	5150.000000	104.000000	60.500000	5.840000	2330.000000
4	4.133333	176.000000	6.793333	156.666667	64033.333333	2.468000	81.433333	1.380000	57566.666667

**MERCI POUR
VOTRE ECOUTE
AVEZ VOUS DES
QUESTIONS?**

TITIEN.CARELLAS@EPITECH.EU

